

Token-Accounting Integrity: Measuring Client-Side Under-Payment in LLM Metering Architectures

(anonymized for review)

Abstract

Usage-based billing is a security boundary in LLM inference: value is streamed to the client before accounting is necessarily finalized, and the charge depends on a multi-category usage record produced at runtime. We study client-side under-payment under an honest-provider threat model, and model this accounting-security boundary along three dimensions — **state synchronization**, **commitment timing**, and **usage authority** — unified by a single integrity property, $V(r) \leq N(r)$, over a request-lifecycle model. *We do not claim new attack primitives*: the enabling mechanisms are individually known, and the contribution is a systematic measurement and architectural analysis of them across representative metering designs, together with an ablation of the enforcement primitives. The three dimensions behave differently under load, and we distinguish empirical from analytic results: the known synchronization race is *created* by contention for shared state and grows with concurrency (slope 0.099 per additional concurrent request, $R^2 = 1.0000$); commitment-timing leakage is *empirically* unchanged across concurrency levels 1–100 under fixed request volume, even though delivery timing itself varies; and usage-authority leakage is *analytically* independent of concurrency under the modelled per-request billing function, which our experiments confirm rather than discover. For commitment timing, an ablation shows that abort-safe finalization closes accounting-integrity violations while reservation independently provides solvency, and that neither substitutes for the other. For usage authority, computing an accurate server-side recount is insufficient unless that value becomes the billing basis: an architecture that recounts correctly, records the result, and still bills the client-declared number leaks 58.3% of the delivered value. Server-authoritative billing eliminates the observed usage-authority leakage. To test whether these are artifacts of one execution or storage design, we repeat the experiments across three topologies (single worker; a reverse proxy over four workers; a load balancer over two gateway instances) and two accounting backends (a mutable balance row; an append-only ledger with a derived balance): 96 accounting cells match the single-worker control with no mismatches, and the eight cases that genuinely exercised both backends agreed. We also state model-level sufficiency conditions per mechanism, separating what is sufficient in the model from what is implementation-specific. Tokenizer cost is real, grows with input length, and varies by $2.9\text{--}3.6\times$ across engines; in the tested local SmolLM2-135M CPU configuration the recount accounted for well under 0.1% of end-to-end inference time (0.014–0.028% across generation lengths). All reported results regenerate from raw data through fail-closed integrity gates, and every headline accounting figure has been independently recomputed from raw data by code that shares nothing with the measurement pipeline.

1 Introduction

Because the billed unit — the token — is produced by opaque, streamed, long-running inference, the pipeline that authorizes, meters, and commits a charge is security-relevant. Recent work studies dishonest *providers* inflating that accounting [1, 2, 3], economic attacks that drain a victim’s budget [4, 5], and dishonest *intermediaries* that misroute or forge provenance [10, 11]. All three assume

Table 1: Adversary directions in LLM inference accounting. Prior work covers the first three; this paper studies the fourth.

Adversary	Goal	Representative work	Studied here
Provider	over-charge an honest client	CoIn [1], Invisible Tokens [2], Token Inflation [3]	no
Third party	inflate a <i>victim</i> ’s bill	Denial-of-Wallet [4], LLMjacking [5]	no
Intermediary	misroute / forge provenance	AEX [10], gateway-path provenance [11]	no
Client	under-pay for received inference	—	yes

the client is honest, impersonated, or irrelevant. We study the remaining direction (Table 1): a legitimate but dishonest client that under-pays.

We are deliberately adversarial about our own contribution. Each enabling mechanism is already known in isolation: a deployed LLM gateway refunds pre-consumed quota when a final usage record is missing after a client disconnect [6]; “never trust client input in a security decision” is CWE-807, and a metered-token proxy already mitigates an encoding-induced “silent zero-debit” [7]; concurrent quota-decrement races are a classic web-race primitive with a recent CVE [8, 9]; and practitioner guidance already warns that retries and disconnects must not cause token leakage [12]. Accordingly, **M1 and M2 are systematization and measurement contributions, and B0 is a known baseline**; we make no priority claim.

Contributions.

1. **An accounting model** (§4) connecting inference execution, value delivery, authorization, reservation, debit, reconciliation and refund in a request-lifecycle state machine, with a formal integrity property that admits legitimate refunds.
2. **An architecture taxonomy** of three experimentally distinguishable dimensions — synchronization (B0, known baseline), commitment timing (M1), and usage authority (M2) — with the known/systematized/defensive boundaries stated explicitly.
3. **A reproducible, cross-architecture measurement study**:
a testbed and methodology quantifying attack success, leakage, leakage efficiency, invariant violations, latency and throughput — repeated across three execution topologies and two accounting backends, with integrity gates shown to fail closed under fault injection, and an independent re-implementation of the strongest defense.
4. **Defense sufficiency conditions and their integrity/performance cost**: model-level sufficient conditions per mechanism (§9), an ablation identifying which primitive closes which property, and recount cost measured at three levels (standalone tokenizer, gateway, and a local real-model experiment).
5. **A machine-checked formalization** (§8): the lifecycle encoded in TLA+ and checked exhaustively with TLC across 10 architecture configurations and 4 invariants, yielding counterexample traces that match the measured failures and an exhaustive orthogonality result for the M1 defenses.
6. **External validity attacked three ways** (§7.4): three execution topologies, three accounting families including an asynchronous event-and-worker pipeline, and a third-party serving stack in place of our own generator.

2 Threat Model and Accounting Invariant

Honest provider; dishonest client holding valid credentials. The client controls exactly six things, each exercised by our harness: request payloads including any client-declared usage fields; permitted headers; request concurrency; the connection lifecycle (it may abort at any moment); retries; and observation of its own responses and balance. It *cannot* compromise TLS, access the database or Redis, read the gateway filesystem, compromise the inference backend, forge authenticated upstream metadata, or alter server code. This is a concrete application-layer adversary, not a symbolic cryptographic one. Every leak we report is therefore attributable to the provider’s own accounting design rather than to a compromise.

Attacker knowledge (disclosed). Our M2 harness constructs each declared-usage manipulation by transforming the *true* server-side usage vector, which grants the attacker more knowledge than a real client has. We therefore grade every manipulation by the knowledge it actually requires: **K0**, client-observable only (the prompt it sent, the response it received, its own balance); **K1**, provider-reported usage (whatever usage the architecture returns to the client); **K2**, oracle access to the exact server-side usage vector. Under-reporting output or input, shaving by a small constant, and declaring an inconsistent total are all **K0**: the client can count its own prompt and the tokens it received. Inflating the cached-input category is **K1**, since the cached/uncached split is not visible in the response. Dropping hidden reasoning tokens is **K2**, because that count is never exposed. The headline usage-authority result — a correct-but-unauthoritative recount leaking 58.3% of delivered value under 90% output under-reporting — rests on a **K0** manipulation and does not depend on oracle knowledge. The two non-K0 manipulations are reported as such and are not used to support the main claim.

For a request r , let $V(r)$ be the value actually delivered, $D(r)$ the committed debits, $F(r)$ refunds, and $N(r) = D(r) - F(r)$ the net committed debit. Under-payment and the integrity property are

$$\text{Leak}(r) = V(r) - N(r), \quad \text{Integrity}(r) : V(r) \leq N(r).$$

The property is an inequality on the *net* debit precisely so that legitimate reconciliation is admitted: a client that consumes less than a reservation must be refunded the difference. A refund is legitimate only when it releases the unconsumed portion, $F(r) \leq \text{Res}(r) - V(r)$; the vulnerable “refund the whole reservation on abort” architecture violates exactly this side condition.

Two distinct properties. Accounting integrity is not the only thing a metering system must provide, and conflating the two obscures the defense analysis. We therefore separate:

Accounting integrity $V(r) \leq N(r)$ — the client is never net-charged less than the value it received.

Solvency a request is served only when the system holds sufficient committed or reserved economic capacity to cover it under the system’s policy.

These are orthogonal. A system can bill exactly for what it delivered while still serving clients who cannot pay (integrity holds, solvency fails), or hold funds and then release them (solvency machinery present, integrity fails). In the M1 dimension, *reservation addresses solvency* and *abort-safe finalization addresses accounting integrity*; §7.2 shows empirically that neither substitutes for the other (Table 13).

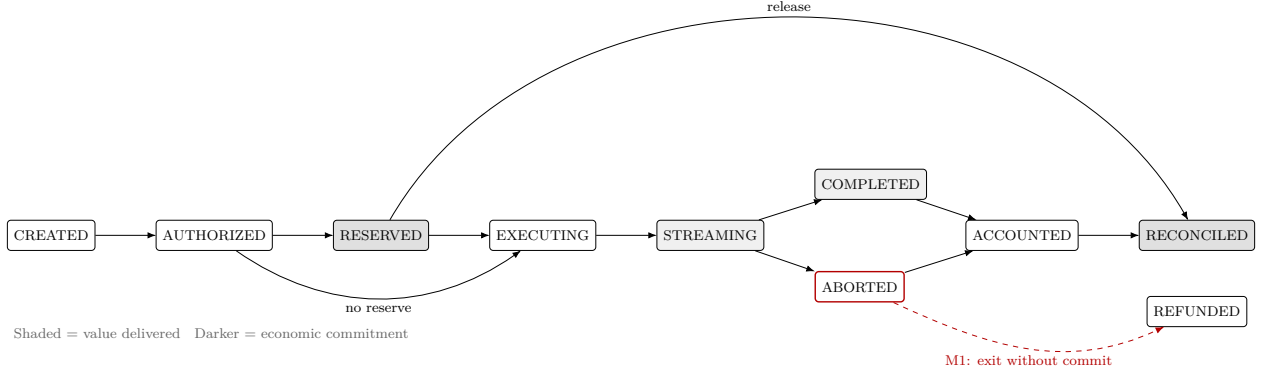


Figure 1: Request lifecycle. Value exposure begins at the first delivered token (STREAMING); economic commitment is final only at RECONCILED. The interval between them is the exposure window that M1 exploits. B0 corrupts the AUTHORIZED→RESERVED transition (non-atomic); M2 corrupts the *data* used at ACCOUNTED.

3 Background and Related Work

LLM APIs meter multi-category token vectors (input, output, cached, reasoning), stream output incrementally, and settle charges asynchronously. Provider-side auditing work establishes that hidden-token accounting can be inflated and is hard to verify [1, 2, 3]; Denial-of-Wallet and LLM-jacking establish victim-side economic exposure [4, 5]; attestation work establishes intermediary-side provenance [10, 11]. Generic quota races are well understood [8, 9]. What we found no prior study of, *in our targeted corpus*, is an evaluation of client-side under-payment under an honest-provider threat model across representative metering architectures with a common measurement and an ablation of the enforcement primitives. This is a bounded negative from a targeted search; it does not establish non-existence.

4 Accounting State Model

We model a metered inference request as a state machine so that each mechanism can be localized to a specific transition, establishing that the taxonomy is *architectural* rather than a list of unrelated implementation bugs.

4.1 Formal model

For a request r let $U_{\text{true}}(r)$ be the usage actually produced, $U_{\text{bill}}(r)$ the usage billed, and $P(\cdot)$ the (exact-decimal) pricing function. Write $C(r) = P(U_{\text{true}}(r))$ for the true cost, $V(r)$ for the value actually delivered to the client, $\text{Res}(r)$ for any reservation, $D(r)$ for committed debits, $F(r)$ for refunds, and

$$N(r) = D(r) - F(r)$$

for the *net committed debit*. Under-payment and the integrity property are

$$\text{Leak}(r) = V(r) - N(r), \quad \text{Integrity}(r) : V(r) \leq N(r).$$

Table 2: Structural distinction between the three mechanisms.

Mechanism	Defective element	Needs concurrency?	Closed by
B0	transition atomicity	yes (≥ 2)	atomic compare-and-decrement
M1	transition graph (abort path)	no	abort-inclusive finalization
M2	data authority at ACCOUNTED	no	server-authoritative recount

Because legitimate reconciliation exists, integrity is an inequality on the *net* debit; a refund is legitimate only when it releases the unconsumed portion,

$$F(r) \leq \text{Res}(r) - V(r).$$

At trial level the harness independently verifies conservation,

$$\sum_{r \in T} N(r) = B_{\text{initial}} - B_{\text{final}},$$

and fails the build on any divergence.

4.2 Mechanism localization

B0 (synchronization). If two concurrent requests read the same authorization state B and both commit a decrement derived from it, one decrement is lost: $\sum_i N(r_i) < \sum_i V(r_i)$. This requires ≥ 2 concurrent transactions, which is why B0 leaks nothing at concurrency 1.

M1 (commitment timing). If $V(r) > 0$ in STREAMING but the exit path taken (ABORTED) does not reach RECONCILED with $N(r) \geq V(r)$, then $\text{Leak}(r) = V(r) > 0$. The defect is in the transition graph, not in a shared resource, hence it is concurrency-invariant.

M2 (usage authority). If the billing basis is client-controlled, $U_{\text{bill}} = T(U_{\text{true}})$ for an attacker transformation T , then $\text{Leak}(r) = P(U_{\text{true}}) - P(T(U_{\text{true}})) > 0$ whenever $P \circ T < P$. Authority is a per-request data-provenance property, hence also concurrency-invariant.

This predicts the empirical result: B0 leakage is *created* by concurrency, while M1/M2 per-request leakage is invariant to it.

5 Taxonomy and Testbed

The three dimensions are experimentally distinguishable, not merely conceptual: **B0** (synchronization) is a non-atomic authorize-then-decrement, is *concurrency-created*, and is labelled a **known baseline** used to validate the measurement rig and to provide a generic reference point; **M1** (commitment timing) is lifecycle-sensitive; **M2** (usage authority) is authority- and pricing-sensitive.

On LLM specificity. We do *not* claim these mechanisms are uniquely possible in LLM systems. Streaming responses, trusting client-supplied values, and racing a counter all occur elsewhere. What is specific to LLM inference is that a single request combines, in one metering boundary: progressive (streamed) value delivery; *runtime-generated* usage that is not known when the request is authorized; multi-category token vectors (input, output, cached, reasoning); tokenizer- and provider-dependent usage semantics; pricing functions defined over those usage categories; and asynchronous reconciliation or settlement. The mechanisms are therefore not claimed as unique

security primitives; rather, LLM inference combines the lifecycle, usage-authority and token-vector accounting conditions into one boundary, and it is that combination we systematize and measure. On this basis B0 remains a *generic* baseline — a shared-counter race that behaves identically in any metered API — while M1 and M2 are LLM-relevant accounting dimensions: M1 is only interesting where value is delivered progressively over a long-running request, and M2 only where the billed quantity is a runtime-produced, multi-category vector priced by a category-sensitive function.

The testbed is a FastAPI gateway over PostgreSQL 16 (READ COMMITTED) and Redis 7, with a deterministic mock generator for reproducibility and *real* tokenizers on the recount path, run under Docker Compose on one commodity host (16 logical CPUs, 16 GB). Money is exact decimal. Five M1 architectures and six M2 architectures share common interfaces so that experiments compare *semantics* rather than unrelated code.

6 Experimental Methodology

Category-wise pricing over three tiers; leakage *efficiency* (fraction of true value evaded) is the price-invariant headline metric and absolute dollars are labelled scenario values. Sweeps: B0 concurrency $\{1, 2, 5, 10, 20, 30, 50\} \times 30$ repetitions; M1 architecture $\times$ abort position $\times$ concurrency $\{1, 5, 20, 50, 100\}$; M2 architecture $\times$ manipulation $\times$ concurrency. Request volume is held *constant* across concurrency levels so that volume cannot masquerade as vulnerability.

Integrity gates. Every summarizer independently recomputes each record’s leak from primitives, checks invariant-flag consistency, and verifies ledger conservation $\sum_r N(r) = B_{\text{init}} - B_{\text{final}}$ without trusting the application’s own bookkeeping; a violation fails the build. We validate the gates adversarially: corrupting a leak value, a debit, a served flag, a balance, an invariant flag, or deleting a ledger row must each be detected (12/12), alongside metamorphic properties (10/10). The metamorphic suite also caught one of our own unjustified assumptions — we had asserted leakage efficiency was price-tier invariant, but that holds only under *uniform* price scaling, not under a structural change to the price ratios — which we corrected rather than suppressed.

Independent verification. The strongest M2 defense is re-implemented in a module sharing no code path with the gateway; verdicts agree on all cross-validated cells (42 M2, 16 M1, 2 B0). The independent checker initially reported three disagreements, which were *definitional*, not defects: it aggregated under-payment only, while the gateway aggregates the signed billing delta. Under flat-total pricing an honest client is systematically *overcharged*, so the two aggregates legitimately differ. We now report both, and use under-payment for the security property.

Statistics. All statistics are produced by a standalone script (scipy/statsmodels) which writes a single output file that the paper cites. Many conditions here are deterministic by construction and have exactly zero variance; for those we report the constant and complete separation rather than a p-value, and reserve nonparametric tests and bootstrap intervals for quantities that genuinely vary.

7 Results

7.1 B0 — synchronization (known baseline)

The non-atomic check-then-decrement leaks only when concurrency ≥ 2 : mean \$-leak is 0.000 at $c=1$, then 0.099, 2.871 and 4.851 at $c=2, 30$ and 50 , growing exactly linearly (slope 0.099 per

additional concurrent request, $R^2 = 1.0000$, $p = 1.9 \times 10^{-50}$). The hardened atomic compare-and-decrement leaks \$0 at every level (Table 10, Fig. 5). B0 is not a contribution; it validates the rig and provides the contrast used below.

7.2 M1 — commitment timing

Leakage tracks the value delivered before the missing commit, rising monotonically with abort position and collapsing to zero at completion when the debit finally fires: per request 0.0045 at ~ 1 token delivered, 0.0285 at 25%, 0.0540 at 50%, 0.0960 at 90%, and 0.0000 at 100% (Fig. 2, Table 11). Request-ASR is 1.00 at every abort position before completion. The two vulnerable architectures — “debit only after completion” and “refund the whole reservation on abort” — are economically identical.

M1 comprises two orthogonal security properties. *Property A (solvency):* served value must not exceed the affordable balance. *Property B (accounting finality):* delivered value must not exceed the net committed debit. The ablation separates them (Table 13). With a budget for two requests and six issued, `no_reserve_settle` preserves accounting integrity (leak 0.0000 at every abort position) yet serves all six and drives the balance to -0.32 ; conversely `reserve_refund_on_abort` holds a reservation but refunds it away, failing both. **Reservation and abort-safe finalization are orthogonal controls; neither substitutes for the other.**

Concurrency is not the mechanism (empirical). Under fixed request volume in the tested architecture, per-request leakage remained unchanged across concurrency levels 1–100: leak per request measured 0.0528 at $c = 1, 5, 20, 50$ and 100 (identical to four decimal places), and request-ASR was exactly 1.000 (vulnerable) or 0.000 (safe) at every level (Table 12, Fig. 3). The honest-client control arm leaked zero everywhere. We do not claim invariance as a proven property of all systems under all loads; we report the measured behaviour of this architecture and, in §11, the structural reason the state model predicts it.

7.3 M2 — usage authority

Writing U for the true usage vector, P for the pricing function and T for the attacker’s transformation of the declared usage, a manipulation pays off exactly when $P(T(U)) < P(U)$ — that is, when the pricing basis *reads the field T changes*. Table 18 derives this per manipulation: a total/subtotal mismatch is inert under category pricing (which never reads the declared total) and is the strongest attack under flat-total pricing (0.741), while reclassifying output into a discounted cached category is the reverse. The exploitable set is a property of the billing function, not of attacker ingenuity. Every server-authoritative architecture drives leakage efficiency to 0.000 across all manipulations.

M2’s concurrency-invariance is *analytic*, not an empirical discovery. The billed amount is $P(T(U))$, a pure function of one request’s declared usage: in our implementation the billed cost is computed before any balance is read, so no shared mutable state enters the billing decision, and the leak of request r cannot depend on what other requests are in flight. The concurrency sweep (Table 17) therefore *confirms* the derivation rather than discovering the invariance — it functions as a check that the implementation contains no unintended shared-state dependency, and it passes. Reported this way, it is a much weaker claim than B0’s measured concurrency slope, and we do not present the two as comparable findings.

Usage computation is not usage authority. This is the central M2 result. The decisive case is `client_logged`, which *computes* a correct server-side recount, *records* it, and nevertheless continues to use the client-declared number as the billing basis. Against a 90% output under-report it leaks at efficiency **0.583** — indistinguishable from the `client` architecture that never recounts at all (Table 16). The ablation reproduces the effect independently with a different probe prompt, at 0.595 (Table 21); the two differ only because efficiency depends on the prompt’s token mix, and both show the recount changing nothing. The implication is that

$$\text{correct usage computation} \not\Rightarrow \text{correct billing},$$

and the architectural property actually required is that *the authoritative usage value used for billing must be derived from a trusted source*. Our ablation demonstrates that recounting without authority does not prevent the observed leakage; in the evaluated `client_logged` architecture the recount provides evidence usable for detection or post-hoc reconciliation (level D1, §7.6) but does not prevent the leak. We do not claim this holds for every conceivable post-hoc enforcement design — a system that acted on the recorded discrepancy before settling would behave differently — only that computing the correct number is not by itself sufficient.

Economic sensitivity. Leakage efficiency is exactly invariant under uniform price scaling ($\times 10$) and shifts by at most 0.012 under a price-*structure* change, so the qualitative conclusions are price-independent (Table 19). A side effect of the same asymmetry is that flat-total pricing systematically *overcharges* honest clients (efficiency -0.058).

Concurrency does not amplify these failures. Increased concurrency is not itself a sufficient amplifier of client-side metering evasion. In the tested architecture, B0 exhibited a concurrency-created race, whereas M1 and M2 remained primarily architectural accounting failures. This negative result follows a chain we can state explicitly: the *state model predicts* that a defect in transition *atomicity* depends on contention for shared state, while defects in the transition *graph* (M1) or in *data authority* (M2) are properties of an individual request; that prediction yields the architecture-level *hypothesis* that only B0 should scale with concurrency; the *controlled measurement* holds request volume constant so that volume cannot masquerade as vulnerability; and the *observed distinction* matches. Concretely: B0 is concurrency-sensitive because shared authorization/debit state is raced; M1 is concurrency-independent in the tested design because the violation lies in the abort path of an individual lifecycle; and M2 is concurrency-independent in the tested design because the violation lies in the authority assigned to per-request usage data. Operationally this means an operator cannot infer safety from low traffic, and cannot detect M1/M2 by watching for load spikes.

7.4 Cross-architecture validation

A measurement study on one deployment invites the objection that its conclusions are artifacts of that deployment. We therefore repeated the experiments across three execution topologies and two accounting backends, changing *only* the deployment or the storage architecture — the attack harness, the architectures under test, and the integrity gates are byte-identical across all runs.

Topologies. (i) *single*: one uvicorn worker, no proxy (the control); (ii) *multi-worker*: nginx over **four** uvicorn workers, i.e. four independent event loops sharing one PostgreSQL and one Redis; (iii) *distributed*: an nginx load balancer over **two separate gateway containers**, two workers each,

again sharing one datastore. Every ledger row records the serving process, so we can confirm load actually spread rather than assuming it: four distinct worker processes served M1 requests in the multi-worker run (101/164/79/136 requests), and both containers served traffic in the distributed run.

Across **96** accounting cells compared against the control there were **no mismatches** (Table 14). Differences are either exactly zero or bounded by the price of a single delivered token — the disconnect-detection jitter already characterized in our reproduction audit — and never change a verdict. B0 is also preserved: the hardened posture leaks exactly \$0 at every concurrency in every topology, *including across two separate containers*, because atomicity is enforced by the database rather than by process locality; and the distributed vulnerable posture reproduced the control exactly (\$0.8910 at $c=10$, \$4.8510 at $c=50$).

A multi-worker deployment is not free. Building the topology exposed two real defects that a single-worker study cannot reveal, and we report them as findings rather than silently fixing them. First, the B0 runtime posture was *per-process* state, so a configuration change would have reached only one of four workers; it must live in shared state (we moved it to Redis). Second, four workers racing schema creation at boot crashed with a uniqueness violation on `pg_class`; startup must be serialized (we use a PostgreSQL advisory lock). Neither changes the accounting semantics, but both are prerequisites for the results above to be meaningful.

Accounting backends. To test whether the conclusions depend on *how money is stored*, we implemented a second backend behind the same semantic interface. Backend A keeps a mutable `credits.balance` row and obtains atomicity from a conditional `UPDATE ... WHERE balance >= cost`. Backend B never mutates that row: every movement is an immutable row in an append-only ledger and the balance is *derived* as the sum of those entries, with the atomic reserve becoming a conditional append serialized on an anchor row. Running the strongest B0/M1/M2 cases against both, all 17 comparison cells agreed — same leakage, same leakage efficiency, same invariant-violation counts, same reconciliation verdicts (Table 15). We state plainly what that number does and does not contain. An audit of our own artifact established that only **8** of the 17 cells genuinely exercised both storage backends; 5 agree for a reason that is analytic rather than experimental (the M2 billed amount is computed before any balance is read, so it cannot depend on the backend); and 4 are *vacuous*, because they are B0 cells and the frozen B0 debit module never routes through the backend abstraction at all — those cells ran identical code twice, so their agreement was guaranteed a priori and carries no information. The honest statement is therefore: *eight genuine backend-sensitive cases agreed, and no case disagreed*. In particular, the frozen B0 debit module contains no reference to the backend abstraction at all, so B0 was not re-executed against Backend B; B0’s storage independence is untested and we list it in §12. Within that scope the result still supports the structural claim — the outcomes follow from where synchronization, commitment and authority sit — but it supports it over eight cases, not seventeen.

A real serving stack. Every figure above was produced against a generator we wrote. That is what makes the accounting arithmetic exact, but it leaves an obvious objection open, so we repeated the experiments with the token source replaced by `llama.cpp`’s `llama-server` running SmolLM2-135M-Instruct on CPU — a widely deployed self-hosted serving stack we did not write, with a real BPE tokenizer, real SSE streaming, and usage records the server produces itself. The gateway, architectures, accounting backends, settlement logic and integrity gates are unchanged; only the token source differs, and the mock default remains byte-identical (the regression suites

still pass 16/16 and 19/19).

Across **12 M1 cells** and **48 M2 cells** there were **zero safety disagreements** (Tables 3, 4). Vulnerable commitment-timing architectures still leak on abort and collapse to zero at completion; `reserve_reconcile` still leaks nothing; every server-authoritative architecture still leaks exactly 0.000 across all eight manipulations.

What *did* move is attack effectiveness, and we report it rather than average it away. Output under-reporting at 90% yields 0.690 here versus 0.583 on the mock, because SmolLM2 emits no reasoning tokens and output therefore dominates the cost. For the same reason `drop_reasoning` — our only K2 manipulation — becomes **completely inert**: there is nothing to drop. Under flat-total pricing, three cells flip sign, because on this token mix flat-total over-charges by so much that even a 90% under-declared total still exceeds the authoritative category cost. **The architectural conclusion is generator-independent; the per-manipulation magnitudes are not.**

The provider’s own number is not reproducible. One property of real serving appeared that the mock cannot exhibit. `llama-server` reports a prefix-cache split (`cached_tokens`), and because cached input is priced an order of magnitude below uncached input, the *honest* charge for a byte-identical request depends on server cache state. Repeating identical requests, the reported split moved (e.g. 24 then 52 of 53 prompt tokens) and the authoritative cost moved with it by 22–32% on 3/3 prompts (Table 5). Three consequences follow directly, and they sharpen the threat model rather than decorate it: the provider’s authoritative number is not reproducible across calls; a client cannot verify its own bill even in principle; and a client that over-declares cached tokens has genuine plausible deniability, because the true value legitimately varies. This is measured on one stack and we do not generalize it to commercial providers — but it is exactly the kind of accounting nondeterminism that distinguishes LLM metering from metering a fixed-price API call.

Asynchronous accounting. Both backends so far settle before the response completes. Real metering pipelines often do not: the gateway emits a usage *event* and a separate worker applies it later. We therefore added a third accounting family — an event queue with a *continuously running* worker and a controlled reconciliation delay D — because a claim of architectural generality that quietly assumes strong consistency is not worth much.

With an honest pipeline, asynchronous accounting is **late, not lossy**: the mean exposure window between value delivery and durable debit tracks D closely ($D = 0, 10, 50, 100, 500$ ms $\rightarrow$ 14.0, 14.9, 54.3, 103.7, 504.1 ms) and the leak reconciles to exactly zero (Table 6). Duplicated events are suppressed idempotently; a *lost* event leaks permanently. Whether a straggler looks lost or merely late is a property of the observation horizon, not of the architecture, and the table says so.

The B0 result is the one that changes an architectural conclusion. We issue requests **strictly sequentially**, 20 ms apart, so no two are ever in flight together — any over-serving therefore *cannot* be a concurrency race. Over-serving nonetheless appears once D reaches 100 ms and reaches **4 requests over a 2-request budget** at $D = 500$ ms, with the balance driven negative (Table 7). **Decoupling settlement converts a timing race into an architectural window: an atomic guarded decrement — the entire B0 defense — provides nothing if it lands after the next authorization has already read the balance.** The sufficiency condition of §9 must therefore be read as requiring atomicity *on the path that authorizes*, not merely somewhere in the pipeline.

M2, by contrast, is untouched: leakage efficiency is identical (0.5823) at every delay, confirming the analytic independence of usage authority from settlement timing.

Table 3: M1 against a real serving stack (llama.cpp llama-server, SmolLM2-135M-Instruct, CPU). Same gateway, same architectures, same accounting; only the token source changed. Vulnerable architectures still leak on abort and collapse to zero at completion; reserve+reconcile still leaks nothing. ‘pre_debit’ over-charges an aborting client, which is a separate defect discussed in the economic analysis.

architecture	abort	n	tokens delivered	leak (\$)	violations	verdict	matches mock
post_completion	abort50	5	24	+0.212500	5	leaks	yes
post_completion	abort90	5	43	+0.355000	5	leaks	yes
post_completion	complete	5	48	+0.000000	0	no leak	yes
reserve_refund_on_abort	abort50	5	24	+0.212500	5	leaks	yes
reserve_refund_on_abort	abort90	5	43	+0.355000	5	leaks	yes
reserve_refund_on_abort	complete	5	48	+0.000000	0	no leak	yes
reserve_reconcile	abort50	5	24	+0.000000	0	no leak	yes
reserve_reconcile	abort90	5	43	+0.000000	0	no leak	yes
reserve_reconcile	complete	5	48	+0.000000	0	no leak	yes
pre_debit	abort50	5	24	-0.180000	0	overcharges	yes
pre_debit	abort90	5	43	-0.037500	0	overcharges	yes
pre_debit	complete	5	48	+0.000000	0	no leak	yes

7.5 Lifecycle failure modes

Beyond the scripted attack we inject nine lifecycle failures (disconnect at $t=0$ and mid-stream, truncated stream, settle-window race, missing / delayed / duplicated usage metadata, and a recount-engine failure). Every invariant violation falls on a vulnerable architecture; the hardened architectures survive all of them; and a recount-engine failure is *fail-closed* — the request is rejected and nothing is served or billed (Table 22).

7.6 Detectability: a claim we withdraw

An earlier version of this work reported a detectability taxonomy (D0 invisible, D1 post-hoc reconcilable, D2 logged discrepancy, D3 prevented) and claimed that merely *logging* a server-side recount moved a leak from D0 to D1. An independent audit of our own artifact showed that claim was **not supported**, and we withdraw it.

Two defects were found. First, the implementation *assigned* the detectability level from static architecture metadata rather than measuring it — for M1 the level was literally `"D3" if arch.safe else "D0"`, a restatement of a flag we had set. Second, when we replaced that with a classifier that reads *only* the retained ledger evidence and is blind to architecture labels, the claimed distinction disappeared: the `client` and `client_logged` architectures retain *byte-identical* evidence (both store a server-observed usage vector alongside the declared one), so both are equally reconcilable. Across the full dataset, 2,200 records whose stored label was D0 are D1 or D3 on the evidence, and **no record in the study qualifies as D0 at all**.

The root cause is a property of the testbed rather than of the architectures: our harness records ground-truth usage for *every* architecture because it needs that value to compute leakage. A production `client` deployment would not necessarily retain it. Our instrumentation therefore cannot distinguish detectability classes, and we make no detectability claim. We report this as a limitation rather than manufacturing additional instrumentation to rescue the metric; the classifier and its output are released with the artifact.

8 Formal Verification

Measurement establishes that a defect occurs on the schedules our harness produced. It cannot establish that a defended architecture is safe on schedules it did not produce. We therefore encode the request lifecycle of §4 in TLA+ and check it exhaustively with TLC.

Model. One specification covers all three dimensions; each dimension is a Boolean constant, so the safe and unsafe architectures are the *same* state machine under different constants, exactly as the testbed implements them behind one interface: **AtomicDebit** (B0), **Reserves** and **AbortSafe** (M1), and **ServerAuthority** (M2). A fifth constant, **ClientMayAbort**, is a *workload* switch rather than an architecture property; disabling it in the B0 configurations keeps the synchronization counterexample free of commitment-timing noise, mirroring how the experiments hold the other two dimensions at their safe setting.

Four invariants are checked, each the formal counterpart of a gate the summarizers already apply to every experimental record: **ACCOUNTINGINTEGRITY** ($\forall r$ terminal: $V(r) \leq N(r)$, gate G1), **SOLVENCY** (balance ≥ 0), **LEDGERCONSERVATION** (all terminal $\Rightarrow B_{\text{init}} - B_{\text{final}} = \sum_r N(r)$, gate G3), and **REFUNDBOUNDED** ($F(r) \leq \max(0, \text{Res}(r) - V(r))$, the paper’s legitimate-refund condition). Solvency is checked *separately* from accounting integrity; collapsing them would destroy the very distinction the ablation exists to establish.

Method. Each invariant is checked in its own TLC run, because TLC halts at the first violation it finds and checking them jointly would hide that an M2 architecture breaks two of them. The result is a full configuration $\times$ invariant matrix (Table 8): **10 configurations $\times$ 4 invariants = 40 independent runs, 27,526 distinct states**. Every expected outcome is declared in `formal/check.py` *before* the run and the script exits non-zero on any disagreement; **all 40 matched, with 0 disagreements**.

What the counterexamples say. TLC returns a shortest counterexample for each unsafe configuration, and each one reproduces the mechanism we measured (Table 9; full traces are in the artifact under `paper/`).

For **B0**, the 21-step trace is the lost update exactly: both requests read snapshot = 2, both blind-write balance := 2 − 2 = 0, both then deliver and are each charged. Note *which* invariant fails — accounting integrity and the refund bound both **hold**, because every individual record is internally consistent; what breaks is conservation of the shared balance. That is precisely why a per-record audit cannot detect B0 while a conservation check can.

For **M1**, the model checks all four combinations of (reserves, abort-safe), not just the two the implementation ships. Reservation alone yields solvency and violates integrity; abort-safe finalization alone yields integrity and drives the balance to −1; only the conjunction satisfies both. **The orthogonality claim is therefore established by exhaustive exploration, not by the two architectures we happened to build.**

For **M2**, the client-authoritative architecture violates accounting integrity, and *also* the refund bound — with a reservation in place, under-billing surfaces as an over-refund. That the same defect appears in two independent invariants is a consequence of the model rather than something we set out to show; it became visible only because the invariants are checked separately. Turning on **ServerAuthority** restores all four invariants *against the same lying client*: the declared value simply stops being read.

Finally, the three defenses *compose*: with contention, aborts and a lying client all enabled, no invariant is violated on any reachable state, at two and at three concurrent requests.

What this does not establish. Nothing here proves a production implementation safe. The model abstracts away the database, the network, partial failure, asynchronous reconciliation, multi-category pricing and tokenizer disagreement; it is finite (2–3 requests, unit pricing); only safety properties are checked, not liveness; and there is **no refinement proof** from the specification to app/. The correspondence is an argument supported by case-by-case agreement between formal and empirical outcomes, and we do not call it more than that. The complete exclusion list is `paper/scope_of_formal_claims.md`, and every claim in the paper carries one of MODEL-CHECKED, FORMALLY ARGUED, ANALYTICALLY DERIVED, DIRECTLY MEASURED or INFERRED accordingly. We never write “proved” for anything weaker than the first.

The model was wrong first. The expectation discipline earned its keep immediately. Our first specification treated a reservation as a *hold* and set $D = \text{charge at settlement}$, so $N = D - F$ fell below the delivered value and TLC rejected the *safe* configurations. The model was wrong, not the paper: in a reserve-and-reconcile architecture the reservation *is* the committed debit and the refund returns its unused part. We record the failure rather than erase it.

9 Defense Sufficiency Conditions

The ablation shows which primitive is load-bearing empirically; here we state, for each mechanism, conditions that are *sufficient within the accounting model* to guarantee $V(r) \leq N(r)$. These are model-level deductive arguments, **not** machine-checked proofs and **not** universal theorems about deployed systems; we separate what the model forces from what our implementation happens to supply.

M1 — commitment timing. Sufficient: *(M1-a) terminal-path totality* — every terminal path, including client disconnect, server exception and stream truncation, reaches a state where an accounting event for r is committed and durable; and *(M1-b) refund boundedness* — every refund satisfies $F(r) \leq \text{Res}(r) - V(r)$. If a reservation was taken, (M1-b) gives $N(r) = \text{Res}(r) - F(r) \geq V(r)$; if not, (M1-a) forces the terminal event to commit $D(r) = V(r)$. Solvency is a *separate* condition *(M1-c)*: reserve atomically before execution and refuse the request when the reservation fails. `no_reserve_settle` satisfies (M1-a)/(M1-b) but not (M1-c), and measured exactly that — integrity held (leak 0.0000) while the balance went to -0.32 .

M2 — usage authority. Sufficient: *(M2-a) authority* — the usage value used for billing is derived from state the attacker cannot control, i.e. $U_{\text{bill}} = U_{\text{true}}$; and *(M2-b) pricing monotonicity* — P is non-decreasing in each category. Computing U_{true} is *not* sufficient: an architecture that computes it correctly, records it, and bills $P(U_{\text{decl}})$ satisfies “the server recounts” while violating (M2-a), and leaks $P(U_{\text{true}}) - P(T(U_{\text{true}}))$.

B0 — synchronization. Sufficient: *(B0-a)* the check balance $\geq \text{cost}$ and the decrement occur as one operation atomic with respect to competing transactions. Under (B0-a) the authorizations on an account are totally ordered, so no two requests authorize against the same funds. With a single in-flight transaction (B0-a) holds vacuously, which is why B0 leaks nothing at concurrency 1.

Implementation-specific caveats. The model says “committed and durable”; our implementation achieves it with a cancellation-shielded settlement task, and a framework that cancelled that task would satisfy the architecture on paper while violating (M1-a). The `upstream` architecture

satisfies (M2-a) *only because the provider is honest by assumption*. And (B0-a) is a property, not a mechanism: our two backends obtain it by a conditional UPDATE and by an anchor-row lock respectively, which is precisely what §7.4 tests. Full statements, arguments and caveats are in the accompanying `defense_sufficiency` document.

10 Defense Evaluation

Each vulnerable architecture is paired with a server-side primitive: atomic compare-and-decrement (B0), reserve-then-abort-safe-finalization (M1), and server-authoritative recount (M2). Under every defense the invariant-violation rate and leakage efficiency are zero across the full attack catalogue.

What the strongest defense costs. We measure recount at three levels because they answer different questions.

1. **Standalone tokenizer** (Fig. 6): cost is linear in input length (log-log slope 0.999–1.14, $R^2 \geq 0.998$), so it budgets as $\text{tokens} \times \text{a per-token constant}$ ($\approx 0.7 \mu\text{s}/\text{token}$ for tiktoken, $\approx 2.5 \mu\text{s}/\text{token}$ for SentencePiece). At 16,384 tokens the engines span 10.1–41.8ms, a $2.9\text{--}3.6\times$ implementation effect; at a fixed token count, workload changes cost by up to $3.4\times$.
2. **Gateway with a *mock* generator:** recount is nearly free at $\leq 1,024$ tokens ($\geq 0.96\times$ throughput) but costs heavily at 16,384 tokens ($0.05\text{--}0.76\times$ at $c \in \{1, 10\}$). Tokenization time explains this (Spearman $\rho = -0.868$, $p = 1.3 \times 10^{-5}$).
3. **Local real-model experiment:** with real autoregressive generation the recount is well under **0.1%** of end-to-end time across 128-, 1024- and 4096-token contexts (0.014–0.019% at 64 new tokens; an independent re-measurement with separately written timing code, at 32 new tokens, obtained 0.017–0.028%—the same order of magnitude, with the ratio rising as generation shortens, exactly as the model predicts), and measured throughput ratios scatter around 1.0 with signs that flip across workloads, i.e. within a $\pm 10\%$ noise band that is itself $\sim 700\times$ larger than the effect.

The apparent expense in (2) is an artifact of the mock generator costing *nothing*: when generation is free, tokenization is the entire request. We report all three rather than the most convenient one, and we do *not* claim recount is “essentially free”: it is a real, predictable cost that is small *relative to real inference* in the tested configuration.

11 Discussion

Three dimensions of one accounting-security boundary behave differently under load, and the difference is predicted by *which* element of the request lifecycle is defective: transition atomicity (B0) is a shared-resource property and was therefore concurrency-created, whereas the transition graph (M1) and data authority (M2) are per-request properties and showed no concurrency dependence in the tested design. The practical consequences are that (i) an operator cannot use traffic level as a proxy for exposure, (ii) integrity and solvency require different controls, and (iii) in the evaluated architectures a correct recount did not prevent leakage unless it was authoritative for billing. The defenses themselves are ordinary; the contribution is knowing precisely which one is load-bearing and what it costs.

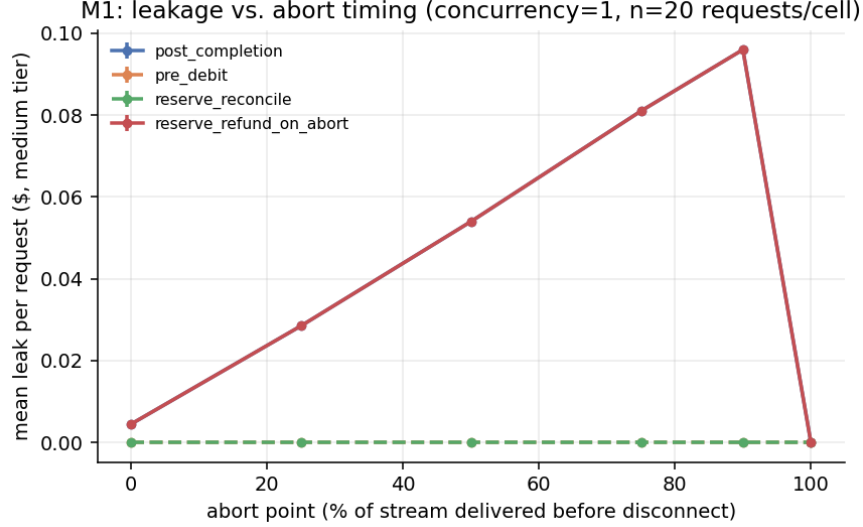


Figure 2: M1: leakage vs. abort position (concurrency 1, medium tier). Vulnerable architectures peak just before the commit and collapse to \$0 at completion; safe architectures stay at \$0. Error bars are within-cell standard deviations.

“Is this not just CWE-807 plus a race plus known streaming bugs?” This is the objection we consider most likely, and it is partly correct — which is why we state it ourselves. B0 *is* a known generic race and is presented as a baseline, not a contribution. The enabling mechanisms behind M1 and M2 are individually known: trusting client-supplied values is CWE-807, and disconnect-related accounting gaps appear in a deployed gateway issue [6], an operations writeup, and practitioner guidance [12]. **We claim no novelty at the primitive level.** What the paper adds is: (i) a unified accounting model in which these otherwise-unrelated failures are the same integrity property violated at different points of one request lifecycle; (ii) a taxonomy that separates synchronization, lifecycle commitment, and usage authority as *experimentally distinguishable* dimensions — the concurrency contrast in §7 is evidence that the separation is real rather than editorial; (iii) an M1 ablation showing that solvency and accounting integrity are orthogonal and require different primitives, which is not derivable from “there is a streaming bug”; (iv) an M2 ablation isolating computation from authority, where an architecture that recounts correctly still leaks 0.583; and (v) a common measurement — economic leakage, leakage efficiency, invariant violations, and defense overhead — applied uniformly across all three dimensions, which no single prior source provides. We do not assert that practitioners believed a recount was sufficient; we show experimentally that in the evaluated architecture it was not.

12 Threats to Validity

Internal. Single uvicorn worker, so gateway recount numbers are an upper bound; shared host and Docker scheduling; local loopback adds a ~ 45 ms floor per request on the Windows/Docker Desktop configuration, which makes the gateway ratios conservative for small inputs; mock generation everywhere except the real-model experiment; leakage that depends on delivered tokens is sensitive to disconnect timing at high concurrency (the within-cell standard deviation of delivered tokens is exactly zero at $c \leq 20$ and ≈ 0.011 at $c \in \{50, 100\}$). The B0 debit path is a frozen module



Figure 3: M1 negative result: with request volume held constant at 20 requests per cell, per-request leakage and request-ASR are invariant to concurrency from 1 to 100.

that predates the pluggable accounting backends and does not reference them, so the backend-parity result covers M1 and M2 only; B0’s independence from the storage representation is *untested*, and the eight genuine backend cells in §7 should not be read as covering it. Our detectability instrumentation was found by our own audit to restate experimenter-assigned labels rather than measure evidence, so we withdraw that claim entirely (§7.6) rather than report a weaker version of it.

Formal. The specification is finite (2–3 concurrent requests, two value chunks, unit pricing) and only safety properties are checked; no liveness property and no parameterized or inductive result is claimed. There is **no refinement proof** from the specification to the implementation — the correspondence is an argument supported by case-by-case agreement, and §8 says so. The model assumes strongly consistent accounting state, so the asynchronous family of §7.4 is *outside* it and its results are labelled empirical only. Multi-category pricing and tokenizer disagreement are likewise unmodelled. The full exclusion list is in the artifact ([paper/scope_of_formal_claims.md](#)).

External validity of the real-stack result. One serving stack, one small model (SmolLM2-135M-Instruct on CPU), one prompt family. It establishes that the architectural conclusions are not artifacts of *our* generator; it does not establish behaviour on commercial providers, and the cache-split nondeterminism we measure is a property of this stack’s prefix cache rather than a claim about hosted APIs. The generator-dependent cells we report — notably that a manipulation targeting hidden reasoning tokens is inert against a model that emits none — bound how far per-manipulation magnitudes travel.

Asynchronous accounting. The reconciliation delay is injected, not observed in the wild; we make no claim about the delays real pipelines run at. The result is conditional and stated as such: *if* settlement lags authorization by more than the inter-arrival time, atomicity on the settlement path does not prevent over-serving.

Two different high-concurrency questions. It is important to separate *accounting-integrity measurements* from *gateway performance measurements*, because only the latter are affected here.

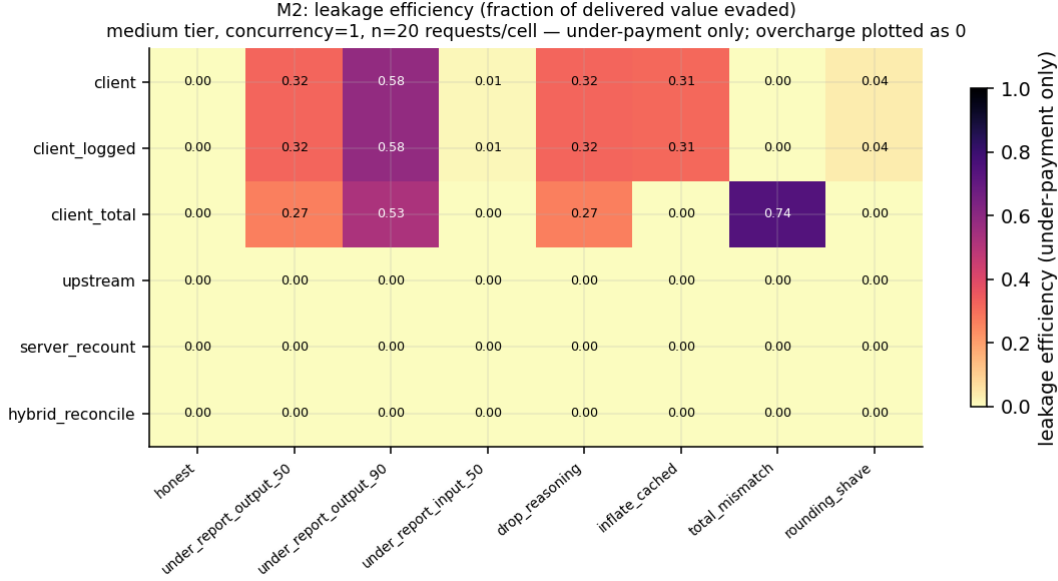


Figure 4: M2: leakage efficiency (fraction of delivered value evaded) by architecture $\times$ manipulation, medium tier, concurrency 1. Server-authoritative rows are uniformly zero; the exploitable set differs between category and flat-total pricing.

(a) *Accounting integrity* was measured at $c = 1, 5, 20, 50$ and 100 and **none of it was discarded**: leakage, leakage efficiency, request-ASR and invariant violations are deterministic functions of the ledger, not of throughput, and every trial passed ledger conservation at every concurrency level.

(b) *Gateway throughput ratios* at $c \geq 50$ are excluded from headline performance claims because the benchmark host became the limiting factor. The repeat spread reaches 17% of the median, the no-recount baseline is itself non-monotonic, and single runs produced ratios > 1 — physically impossible for strictly-added work — which is what motivated the switch to median-of-three repeats. These are symptoms of single-worker server and client/event-loop saturation, not of recount cost. Only $c = 1$ and $c = 10$ are treated as performance evidence; the excluded cells remain published in **results/** and summarized here rather than removed.

Construct. Synthetic price tiers; a token-count abstraction of “inference value”; and leakage normalized by delivered value.

External. The cross-architecture experiments (§7.4) remove the single-event-loop, single-process and single-storage-design confounds, but they do not remove all of them, and we state the boundary precisely. *Addressed*: four independent event loops; two separate gateway containers behind a load balancer; and two structurally different accounting backends (mutable row vs. append-only derived ledger). *Not addressed*: all instances still run on **one physical host** sharing **one** PostgreSQL and **one** Redis, so multi-host networking, replica lag, cross-region latency and datastore partitioning are untested; both backends are strongly-consistent single-datastore designs, so an **eventually-consistent** or sharded accounting pipeline — which could break the atomicity condition (B0-a) or the durability condition (M1-a) — remains unevaluated. We also test one model family, one cache, and no commercial provider; the real-model experiment bounds a *ratio* in one configuration and

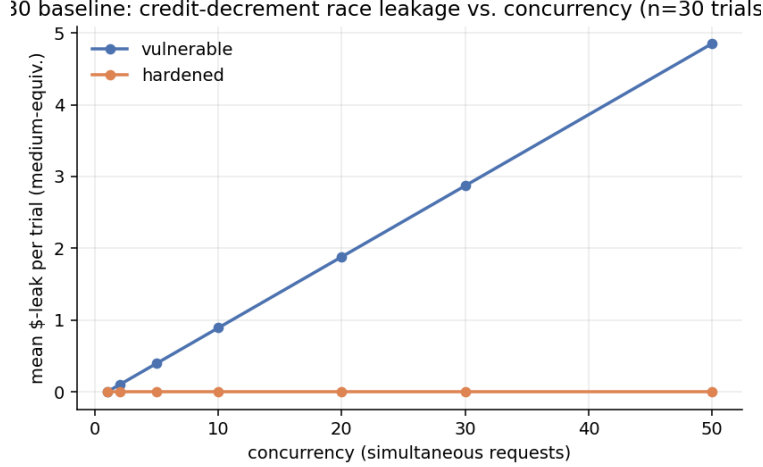


Figure 5: B0 baseline: mean \$-leak per trial vs. concurrency (30 repetitions per cell). Leakage is *created* by concurrency, in contrast with M1/M2.

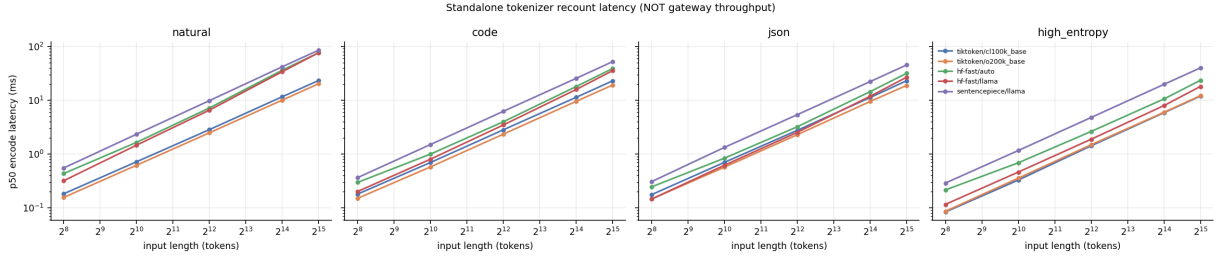


Figure 6: Standalone tokenizer recount latency (p50, log-log) by engine, workload and input length, over 100 exact-length-verified cells. This is *not* a gateway throughput result.

does not validate any commercial provider’s billing behaviour.

Formal. The sufficiency conditions of §9 are model-level deductive arguments, not machine-checked proofs. We do not claim they are sufficient in an arbitrary deployed system, nor that no other sufficient condition set exists. Their *necessity* evidence is empirical: removing a condition produced a measured violation in our implementation.

Literature. A targeted search rather than a proof of absence, in a fast-moving area with terminology mismatch risk.

13 Responsible Disclosure and Ethics

This work produced no new vulnerability findings against any third-party, live, or production system; all experiments ran against the local testbed shipped with the artifact, whose harness defaults to `localhost` and contains no third-party endpoints. No credentials, real user data, or production billing APIs were used. The public artifacts we cite as prior evidence (a GitHub issue [6], a PR [7], and a CVE [9]) were public before this work and are not our findings. Every attack in

the artifact is paired with a defense, and the testbed is released so operators can measure their own gateways.

14 Conclusion

Client-side under-payment in LLM metering is best modelled as one accounting boundary with three placements: synchronization, commitment timing, and usage authority. Localizing each to a state transition predicts — and our measurements confirm — that only the synchronization defect is concurrency-created, that integrity and solvency are closed by different primitives, and that a server-side recount must be authoritative rather than merely computed. In the tested configuration that defense costs a small, predictable fraction of real inference time.

References

- [1] G. Sun et al. CoIn: Counting the Invisible Reasoning Tokens in Commercial Opaque LLM APIs. arXiv:2505.13778, 2025.
- [2] G. Sun et al. Invisible Tokens, Visible Bills. arXiv:2505.18471, 2025.
- [3] S. Hoque et al. Token Inflation: How Dishonest Providers Can Overcharge for LLM Usage. arXiv:2605.30040, 2026.
- [4] OWASP. LLM10: Unbounded Consumption (Denial of Wallet). OWASP Top 10 for LLM Applications, 2025.
- [5] Sysdig Threat Research Team. LLMjacking: stolen cloud credentials used for LLM inference, 2024.
- [6] QuantumNous/new-api issue #5235: streaming requests may refund all pre-consumed quota when final usage is missing after client disconnect, 2026.
- [7] lightninglabs/aperture PR #247: metered draw-down of prepaid L402 tokens; strip `Accept-Encoding` to prevent a silent zero-debit, 2026.
- [8] J. Kettle. The single-packet attack: making remote race-conditions ‘local’. PortSwigger Research, 2023.
- [9] CVE-2026-31873: Tyk API Gateway non-atomic quota check/decrement race, 2026.
- [10] Y. Guan. AEX: Non-Intrusive Multi-Hop Attestation and Provenance for LLM APIs. arXiv:2603.14283, 2026.
- [11] F. Wang and Z. Tian. Evidence-Bound Gateway-Path Provenance for Third-Party LLM Inference. arXiv:2606.22560, 2026.
- [12] Industry metering guidance: gateway as enforcement point vs. meter as accounting point; retries, partial failures and client disconnects must not cause double billing or token leakage, 2026.

Table 4: M2 against a real serving stack. Leakage efficiency differs from the mock corpus because the real token mix differs – notably SmolLM2 reports no reasoning tokens, so output dominates the cost and output under-reporting pays more. Every server-authoritative architecture still leaks exactly zero across all eight manipulations.

architecture	manipulation	real eff.	mock eff.	safe as expected
client	honest	+0.0000	+0.0000	yes
client	under_report_output_50	+0.3766	+0.3237	yes
client	under_report_output_90	+0.6904	+0.5827	yes
client	under_report_input_50	+0.1151	+0.0144	yes
client	rounding_shave	+0.0523	+0.0360	yes
client	total_mismatch	+0.0000	+0.0000	yes
client	inflate_cached	+0.3640	+0.3129	yes
client	drop_reasoning	+0.0000	+0.3237	yes
client_logged	honest	+0.0000	+0.0000	yes
client_logged	under_report_output_50	+0.3766	+0.3237	yes
client_logged	under_report_output_90	+0.6904	+0.5827	yes
client_logged	under_report_input_50	+0.1151	+0.0144	yes
client_logged	rounding_shave	+0.0523	+0.0360	yes
client_logged	total_mismatch	+0.0000	+0.0000	yes
client_logged	inflate_cached	+0.3640	+0.3129	yes
client_logged	drop_reasoning	+0.0000	+0.3237	yes
client_total	honest	-1.0868	-0.0576	yes
client_total	under_report_output_50	-0.7103	+0.2662	yes
client_total	under_report_output_90	-0.3964	+0.5252	yes
client_total	under_report_input_50	-0.7416	-0.0144	yes
client_total	rounding_shave	-1.0241	-0.0144	yes
client_total	total_mismatch	+0.4822	+0.7410	yes
client_total	inflate_cached	-1.0868	-0.0576	yes
client_total	drop_reasoning	-1.0868	+0.2662	yes
server_recount	honest	+0.0000	+0.0000	yes
server_recount	under_report_output_50	+0.0000	+0.0000	yes
server_recount	under_report_output_90	+0.0000	+0.0000	yes
server_recount	under_report_input_50	+0.0000	+0.0000	yes
server_recount	rounding_shave	+0.0000	+0.0000	yes
server_recount	total_mismatch	+0.0000	+0.0000	yes
server_recount	inflate_cached	+0.0000	+0.0000	yes
server_recount	drop_reasoning	+0.0000	+0.0000	yes
hybrid_reconcile	honest	+0.0000	+0.0000	yes
hybrid_reconcile	under_report_output_50	+0.0000	+0.0000	yes
hybrid_reconcile	under_report_output_90	+0.0000	+0.0000	yes
hybrid_reconcile	under_report_input_50	+0.0000	+0.0000	yes
hybrid_reconcile	rounding_shave	+0.0000	+0.0000	yes
hybrid_reconcile	total_mismatch	+0.0000	+0.0000	yes
hybrid_reconcile	inflate_cached	+0.0000	+0.0000	yes
hybrid_reconcile	drop_reasoning	20 +0.0000	+0.0000	yes
upstream	honest	+0.0000	+0.0000	yes
upstream	under_report_output_50	+0.0000	+0.0000	yes

Table 5: The authoritative cost of a byte-identical request on a real serving stack. ‘cached_tokens’ is a prefix-cache split reported by the server; because cached input is priced 10x below uncached input, the provider’s own honest charge for the same request changes with server cache state. The mock generator cannot exhibit this.

prompt tokens	cached tokens per call	honest cost per call	spread
53	24 / 52 / 52 / 52	0.039700 / 0.027100 / 0.027100 / 0.027100	31.74%
53	35 / 52 / 52 / 52	0.034750 / 0.027100 / 0.027100 / 0.027100	22.01%
53	35 / 52 / 52 / 52	0.034750 / 0.027100 / 0.027100 / 0.027100	22.01%

Table 6: M1 under asynchronous settlement with a continuously running accounting worker. With an honest pipeline the mean exposure window tracks the configured reconciliation delay and the leak reconciles to exactly zero: asynchronous accounting is late, not lossy. Duplicate events are suppressed idempotently. A lost event leaks permanently. Whether the ‘delayed_event’ straggler (fixed 1 s) appears lost or merely late depends on the observation horizon (4x the delay, floored at 400 ms), which is a property of the observer, not of the architecture.

delay (ms)	fault	delivered (\$)	charged (\$)	leak (\$)	window mean (ms)	window p95 (ms)	reconciled
0	none	0.325500	0.325500	0.000000	14.0	8.3	yes
0	lost_event	0.306000	0.000000	0.306000	-	-	NO
0	duplicate_event	0.304500	0.304500	0.000000	5.7	6.2	yes
0	delayed_event	0.310500	0.000000	0.310500	-	-	NO
0	abort_before_event	0.330000	0.081000	0.249000	5.3	5.3	NO
10	none	0.282000	0.282000	0.000000	14.9	15.4	yes
10	lost_event	0.319500	0.000000	0.319500	-	-	NO
10	duplicate_event	0.277500	0.277500	0.000000	14.8	15.5	yes
10	delayed_event	0.313500	0.000000	0.313500	-	-	NO
10	abort_before_event	0.289500	0.000000	0.289500	-	-	NO
50	none	0.291000	0.291000	0.000000	54.3	55.3	yes
50	lost_event	0.310500	0.000000	0.310500	-	-	NO
50	duplicate_event	0.321000	0.321000	0.000000	53.9	54.2	yes
50	delayed_event	0.319500	0.000000	0.319500	-	-	NO
50	abort_before_event	0.297000	0.081000	0.216000	54.1	54.1	NO
100	none	0.298500	0.298500	0.000000	103.7	104.3	yes
100	lost_event	0.334500	0.000000	0.334500	-	-	NO
100	duplicate_event	0.262500	0.262500	0.000000	103.8	104.1	yes
100	delayed_event	0.303000	0.000000	0.303000	-	-	NO
100	abort_before_event	0.328500	0.000000	0.328500	-	-	NO
500	none	0.340500	0.340500	0.000000	504.1	504.8	yes
500	lost_event	0.334500	0.000000	0.334500	-	-	NO
500	duplicate_event	0.321000	0.321000	0.000000	504.0	504.0	yes
500	delayed_event	0.340500	0.340500	0.000000	1010.5	1014.9	yes
500	abort_before_event	0.342000	0.162000	0.180000	517.5	514.9	NO

Table 7: B0 under asynchronous settlement. Requests are issued STRICTLY SEQUENTIALLY, 20 ms apart, so no two are ever in flight together: any over-serving here cannot be a concurrency race. Over-serving instead appears once the reconciliation delay exceeds the inter-arrival time, and the balance goes negative. Decoupling settlement converts a timing race into an architectural window.

delay (ms)	inter-arrival (ms)	issued	served	affordable	over-served	final balance (\$)
0	20	6	2	2	0	0.000000
10	20	6	2	2	0	0.000000
50	20	6	2	2	0	0.000000
100	20	6	3	2	1	-0.081000
500	20	6	6	2	4	-0.324000

Table 8: TLC model-checking results. Each cell is an independent exhaustive run; ‘holds’ means the invariant survived every reachable state and ‘VIOLATED’ means TLC returned a counterexample. All 40 cells matched the expectation declared before the run. States = largest reachable state space among the four runs for that configuration.

configuration	mech	Integrity	Solvency	Conservation	Refund bnd	states
b0 vulnerable	B0	holds	holds	VIOLATED	holds	157
b0 hardened	B0	holds	holds	holds	holds	37
m1 post completion	M1	VIOLATED	holds	holds	holds	324
m1 reserve refund on abort	M1	VIOLATED	holds	holds	VIOLATED	361
m1 no reserve settle	M1	holds	VIOLATED	holds	holds	336
m1 reserve reconcile	M1	holds	holds	holds	holds	361
m2 client	M2	VIOLATED	holds	holds	VIOLATED	361
m2 server recount	M2	holds	holds	holds	holds	361
all defenses	ALL	holds	holds	holds	holds	172
all defenses 3req	ALL	holds	holds	holds	holds	4,807

Table 9: Formal-to-empirical correspondence. Every model-checked outcome is paired with the measurement of the implemented architecture it abstracts. The formal model predicts which property fails; the experiment quantifies how much value leaks when it does.

formal configuration	model-checked outcome	measured outcome
b0 vulnerable	Conservation (trace 21)	leak grows with concurrency: slope 0.099/request, $R^2=1.0000$
b0 hardened	all four hold	\$0 leak at every tested concurrency (1..50)
m1 post completion	Integrity (trace 7)	leak = value of tokens delivered before the commit; request-ASR 1.000
m1 reserve refund on abort	Integrity, Refund bnd (trace 7)	leak > 0 at every abort position; invariant violated
m1 no reserve settle	Solvency (trace 12)	accounting integrity holds; final balance goes NEGATIVE
m1 reserve reconcile	all four hold	\$0 leak, 0 invariant violations, balance never negative
m2 client	Integrity, Refund bnd (trace 8)	leakage efficiency 0.583 (prompt A) / 0.595 (prompt B)
m2 server recount	all four hold	leakage efficiency 0.000 across every manipulation
all defenses	all four hold	16/16 regression checks; 60 cross-validation cells agree
all defenses 3req	all four hold	no configuration produced a violation

Table 10: B0 baseline: credit-decrement race (30 reps/cell).

posture	conc	served	\$-leak	trial ASR	req ASR	detect
hardened	1	1.0	0.0000	0.00	0.000	none
hardened	2	2.0	0.0000	0.00	0.000	none
hardened	5	5.0	0.0000	0.00	0.000	none
hardened	10	10.0	0.0000	0.00	0.000	none
hardened	20	10.0	0.0000	0.00	0.000	none
hardened	30	10.0	0.0000	0.00	0.000	none
hardened	50	10.0	0.0000	0.00	0.000	none
vulnerable	1	1.0	0.0000	0.00	0.000	none
vulnerable	2	2.0	0.0990	1.00	0.500	partial
vulnerable	5	5.0	0.3960	1.00	0.800	partial
vulnerable	10	10.0	0.8910	1.00	0.900	partial
vulnerable	20	20.0	1.8810	1.00	0.950	partial
vulnerable	30	30.0	2.8710	1.00	0.967	partial
vulnerable	50	50.0	4.8510	1.00	0.980	partial

Table 11: M1 metering-commit timing: leak vs. abort (medium tier, concurrency=1).

architecture	abort%	delivered	leak/req	req ASR	inv.viol	detect
pre_debit	0	1	0.0000	0.00	0.00	n/a
pre_debit	25	17	0.0000	0.00	0.00	n/a
pre_debit	50	34	0.0000	0.00	0.00	n/a
pre_debit	75	52	0.0000	0.00	0.00	n/a
pre_debit	90	62	0.0000	0.00	0.00	n/a
pre_debit	100	69	0.0000	0.00	0.00	n/a
reserve_reconcile	0	1	0.0000	0.00	0.00	n/a
reserve_reconcile	25	17	0.0000	0.00	0.00	n/a
reserve_reconcile	50	34	0.0000	0.00	0.00	n/a
reserve_reconcile	75	52	0.0000	0.00	0.00	n/a
reserve_reconcile	90	62	0.0000	0.00	0.00	n/a
reserve_reconcile	100	69	0.0000	0.00	0.00	n/a
post_completion	0	1	0.0045	1.00	1.00	D0
post_completion	25	17	0.0285	1.00	1.00	D0
post_completion	50	34	0.0540	1.00	1.00	D0
post_completion	75	52	0.0810	1.00	1.00	D0
post_completion	90	62	0.0960	1.00	1.00	D0
post_completion	100	69	0.0000	0.00	0.00	n/a
reserve_refund_on_abort	0	1	0.0045	1.00	1.00	D0
reserve_refund_on_abort	25	17	0.0285	1.00	1.00	D0
reserve_refund_on_abort	50	34	0.0540	1.00	1.00	D0
reserve_refund_on_abort	75	52	0.0810	1.00	1.00	D0
reserve_refund_on_abort	90	62	0.0960	1.00	1.00	D0
reserve_refund_on_abort	100	69	0.0000	0.00	0.00	n/a

Table 12: M1: leak PER REQUEST vs concurrency (request volume held constant, attacker arm).

architecture	c=1	c=5	c=20	c=50	c=100
post_completion	0.0528	0.0528	0.0528	0.0528	0.0528
pre_debit	0.0000	0.0000	0.0000	0.0000	0.0000
reserve_reconcile	0.0000	0.0000	0.0000	0.0000	0.0000
reserve_refund_on_abort	0.0528	0.0528	0.0528	0.0528	0.0528

Table 13: M1 architecture matrix. Solvency (served value must not exceed the affordable balance) and accounting integrity (delivered value must not exceed the net committed debit) are ORTHOGONAL: reservation provides the former and abort-safe finalization the latter, and neither substitutes for the other.

architecture	Property A: solvency	Property B: accounting integrity	how the violated property fails
pre_debit	HOLDS	HOLDS	-
reserve_reconcile	HOLDS	HOLDS	-
reserve_refund_on_abort	VIOLATED	VIOLATED	served 6/6 on a 2-request budget
no_reserve_settle	VIOLATED	HOLDS	served 6/6 on a 2-request budget; balance -0.320000
post_completion	VIOLATED	VIOLATED	served 6/6 on a 2-request budget

Table 14: Cross-topology validation. Accounting outcomes are compared against the single-worker control. 'diff in tokens' expresses an M1 difference in whole delivered tokens; sub-token differences reflect disconnect-detection timing, not accounting divergence. 'distinct procs' confirms load actually spread.

mech	cell	topology	single	topology value	abs diff	diff in tokens	distinct procs	verdict
M1	post_completion @0%	multiworker	0.0045	0.0047	0.0002	0.15	4	WITHIN 1 TOKEN
M1	post_completion @25%	multiworker	0.0285	0.0285	0.0000	0.00	4	IDENTICAL
M1	post_completion @50%	multiworker	0.0540	0.0540	0.0000	0.01	4	WITHIN 1 TOKEN
M1	post_completion @75%	multiworker	0.0810	0.0810	0.0000	0.00	4	IDENTICAL
M1	post_completion @90%	multiworker	0.0960	0.0960	0.0000	0.00	4	IDENTICAL
M1	post_completion @100%	multiworker	0.0000	0.0000	0.0000	0.00	4	IDENTICAL
M1	pre_debit @0%	multiworker	-0.1020	-0.1019	0.0001	0.09	4	WITHIN 1 TOKEN
M1	pre_debit @25%	multiworker	-0.0780	-0.0780	0.0000	0.00	4	IDENTICAL
M1	pre_debit @50%	multiworker	-0.0525	-0.0525	0.0000	0.00	4	IDENTICAL
M1	pre_debit @75%	multiworker	-0.0255	-0.0255	0.0000	0.00	4	IDENTICAL
M1	pre_debit @90%	multiworker	-0.0105	-0.0105	0.0000	0.00	4	IDENTICAL
M1	pre_debit @100%	multiworker	0.0000	0.0000	0.0000	0.00	4	IDENTICAL
M1	reserve_reconcile @0%	multiworker	0.0000	0.0000	0.0000	0.00	4	IDENTICAL
M1	reserve_reconcile @25%	multiworker	0.0000	0.0000	0.0000	0.00	4	IDENTICAL
M1	reserve_reconcile @50%	multiworker	0.0000	0.0000	0.0000	0.00	4	IDENTICAL
M1	reserve_reconcile @75%	multiworker	0.0000	0.0000	0.0000	0.00	4	IDENTICAL
M1	reserve_reconcile @90%	multiworker	0.0000	0.0000	0.0000	0.00	4	IDENTICAL
M1	reserve_reconcile @100%	multiworker	0.0000	0.0000	0.0000	0.00	4	IDENTICAL
M1	reserve_refund_on_abort @0%	multiworker	0.0045	0.0045	0.0000	0.03	4	WITHIN 1 TOKEN
M1	reserve_refund_on_abort @25%	multiworker	0.0285	0.0285	0.0000	0.00	4	IDENTICAL
M1	reserve_refund_on_abort @50%	multiworker	0.0540	0.0540	0.0000	0.00	4	IDENTICAL
M1	reserve_refund_on_abort @75%	multiworker	0.0810	0.0810	0.0000	0.00	4	IDENTICAL
M1	reserve_refund_on_abort @90%	multiworker	0.0960	0.0960	0.0000	0.00	4	IDENTICAL
M1	reserve_refund_on_abort @100%	multiworker	0.0000	0.0000	0.0000	0.00	4	IDENTICAL
M1	post_completion @25%	distributed	0.0285	0.0285	0.0000	0.00	4	IDENTICAL
M1	post_completion @50%	distributed	0.0540	0.0540	0.0000	0.01	4	WITHIN 1 TOKEN
M1	post_completion @90%	distributed	0.0960	0.0960	0.0000	0.03	4	WITHIN 1 TOKEN
M1	pre_debit @25%	distributed	-0.0780	-0.0780	0.0000	0.00	4	IDENTICAL
M1	pre_debit @50%	distributed	-0.0525	-0.0525	0.0000	0.00	4	IDENTICAL
M1	pre_debit @90%	distributed	-0.0105	-0.0105	0.0000	0.00	4	IDENTICAL
M1	reserve_reconcile @25%	distributed	0.0000	0.0000	0.0000	0.00	4	IDENTICAL
M1	reserve_reconcile @50%	distributed	0.0000	0.0000	0.0000	0.00	4	IDENTICAL
M1	reserve_reconcile @90%	distributed	0.0000	0.0000	0.0000	0.00	4	IDENTICAL
M1	reserve_refund_on_abort @25%	distributed	0.0285	0.0285	0.0000	0.00	4	IDENTICAL
M1	reserve_refund_on_abort @50%	distributed	0.0540	0.0540	0.0000	0.00	4	IDENTICAL
M1	reserve_refund_on_abort @90%	distributed	0.0960	0.0960	0.0000	0.00	4	IDENTICAL
M2	client/drop_reasoning	multiworker	0.324	0.324	0.0000	-	4	IDENTICAL
M2	client/honest	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	client/inflate_cached	multiworker	0.313	0.313	0.0000	-	4	IDENTICAL
M2	client/rounding_shave	multiworker	0.036	0.036	0.0000	-	4	IDENTICAL
M2	client/total_mismatch	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	client/under_report_output_50	multiworker	0.324	0.324	0.0000	-	4	IDENTICAL
M2	client/under_report_output_90	multiworker	0.583	0.583	0.0000	-	4	IDENTICAL
M2	client_logged/drop_reasoning	multiworker	0.324	0.324	0.0000	-	4	IDENTICAL
M2	client_logged/honest	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	client_logged/inflate_cached	multiworker	0.313	0.313	0.0000	-	4	IDENTICAL
M2	client_logged/rounding_shave	multiworker	0.036	0.036	0.0000	-	4	IDENTICAL
M2	client_logged/total_mismatch	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	client_logged/under_report_output_50	multiworker	0.324	0.324	0.0000	-	4	IDENTICAL
M2	client_logged/under_report_output_90	multiworker	0.583	0.583	0.0000	-	3	IDENTICAL
M2	client_total/drop_reasoning	multiworker	0.266	0.266	0.0000	-	4	IDENTICAL
M2	client_total/honest	multiworker	-0.058	-0.058	0.0000	-	4	IDENTICAL
M2	client_total/inflate_cached	multiworker	-0.058	-0.058	0.0000	-	4	IDENTICAL
M2	client_total/rounding_shave	multiworker	-0.014	-0.014	0.0000	-	4	IDENTICAL
M2	client_total/total_mismatch	multiworker	0.741	0.741	0.0000	-	4	IDENTICAL
M2	client_total/under_report_output_50	multiworker	0.266	0.266	0.0000	-	4	IDENTICAL
M2	client_total/under_report_output_90	multiworker	0.525	0.525	0.0000	-	4	IDENTICAL
M2	hybrid_reconcile/drop_reasoning	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	hybrid_reconcile/honest	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	hybrid_reconcile/inflate_cached	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	hybrid_reconcile/rounding_shave	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	hybrid_reconcile/total_mismatch	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	hybrid_reconcile/under_report_output_50	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	hybrid_reconcile/under_report_output_90	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	server_recount/drop_reasoning	multiworker	0.000	0.000	0.0000	-	3	IDENTICAL
M2	server_recount/honest	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	server_recount/inflate_cached	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	server_recount/rounding_shave	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	server_recount/total_mismatch	multiworker	0.000	0.000	0.0000	-	3	IDENTICAL
M2	server_recount/under_report_output_50	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	server_recount/under_report_output_90	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	upstream/drop_reasoning	multiworker	0.000	0.000	0.0000	-	3	IDENTICAL
M2	upstream/honest	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	upstream/inflate_cached	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	upstream/rounding_shave	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	upstream/total_mismatch	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	upstream/under_report_output_50	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	upstream/under_report_output_90	multiworker	0.000	0.000	0.0000	-	4	IDENTICAL
M2	client/honest	distributed	0.000	0.000	0.0000	-	4	IDENTICAL
M2	client/total_mismatch	distributed	0.000	0.000	0.0000	-	4	IDENTICAL
M2	client/under_report_output_90	distributed	0.583	0.583	0.0000	-	4	IDENTICAL
M2	client_logged/honest	distributed	0.000	0.000	0.0000	-	4	IDENTICAL

Table 15: Cross-backend validation. The same cases run against a mutable balance row and an append-only ledger with a derived balance, behind one semantic interface. The ‘evidence’ column states what each agreement is worth: GENUINE = the case actually exercised both backends and could have disagreed; ANALYTIC = the M2 billed amount is computed before any balance is read, so it cannot vary by backend; VACUOUS = agreement between two zero-leak outcomes, which any correct implementation produces. B0 cases are ANALYTIC-at-best because the frozen B0 debit module does not reference the backend abstraction. Only the GENUINE count is evidence of storage-architecture independence.

case	metric	mutable	ledger	viol (mut)	viol (led)	verdict	evidence
B0/hardened/c10	leak	0.000000	0.000000	0	0	IDENTICAL	VACUOUS
B0/hardened/c50	leak	0.000000	0.000000	0	0	IDENTICAL	VACUOUS
B0/vulnerable/c10	leak	0.949500	0.949500	9	9	IDENTICAL	VACUOUS
B0/vulnerable/c50	leak	5.169500	5.169500	49	49	IDENTICAL	VACUOUS
M1/no_reserve_settle/abort50	leak_per_request	0.000000	0.000000	0	0	IDENTICAL	GENUINE
M1/no_reserve_settle/abort90	leak_per_request	0.000000	0.000000	0	0	IDENTICAL	GENUINE
M1/post_completion/abort50	leak_per_request	0.054500	0.054500	10	10	IDENTICAL	GENUINE
M1/post_completion/abort90	leak_per_request	0.095000	0.095000	10	10	IDENTICAL	GENUINE
M1/reserve_reconcile/abort50	leak_per_request	0.000000	0.000000	0	0	IDENTICAL	GENUINE
M1/reserve_reconcile/abort90	leak_per_request	0.000000	0.000000	0	0	IDENTICAL	GENUINE
M1/reserve_refund_on_abort/abort50	leak_per_request	0.054500	0.054500	10	10	IDENTICAL	GENUINE
M1/reserve_refund_on_abort/abort90	leak_per_request	0.095000	0.095000	10	10	IDENTICAL	GENUINE
M2/client_under_report_output_90	efficiency	0.5942492012779552	0.5942492012779552	10	10	IDENTICAL	ANALYTIC
M2/client_logged/under_report_output_90	efficiency	0.5942492012779552	0.5942492012779552	10	10	IDENTICAL	ANALYTIC
M2/client_total/total_mismatch	efficiency	0.7412140575079872	0.7412140575079872	10	10	IDENTICAL	ANALYTIC
M2/hybrid_reconcile/total_mismatch	efficiency	0.0	0.0	0	0	IDENTICAL	ANALYTIC
M2/server_recount/under_report_output_90	efficiency	0.0	0.0	0	0	IDENTICAL	ANALYTIC

Table 16: M2 usage-record authority: leakage efficiency by manipulation (medium tier, concurrency=1).

architecture	manipulation	leak eff.	req ASR	inv.viol	detect
client	honest	0.00	0.00	0.00	n/a
client	under_report_output_50	0.32	1.00	1.00	D0
client	under_report_output_90	0.58	1.00	1.00	D0
client	under_report_input_50	0.01	1.00	1.00	D0
client	drop_reasoning	0.32	1.00	1.00	D0
client	inflate_cached	0.31	1.00	1.00	D0
client	total_mismatch	0.00	0.00	0.00	n/a
client	rounding_shave	0.04	1.00	1.00	D0
client_logged	honest	0.00	0.00	0.00	n/a
client_logged	under_report_output_50	0.32	1.00	1.00	D1
client_logged	under_report_output_90	0.58	1.00	1.00	D1
client_logged	under_report_input_50	0.01	1.00	1.00	D1
client_logged	drop_reasoning	0.32	1.00	1.00	D1
client_logged	inflate_cached	0.31	1.00	1.00	D1
client_logged	total_mismatch	0.00	0.00	0.00	n/a
client_logged	rounding_shave	0.04	1.00	1.00	D1
client_total	honest	0.00	0.00	0.00	n/a
client_total	under_report_output_50	0.27	1.00	1.00	D0
client_total	under_report_output_90	0.53	1.00	1.00	D0
client_total	under_report_input_50	0.00	0.00	0.00	n/a
client_total	drop_reasoning	0.27	1.00	1.00	D0
client_total	inflate_cached	0.00	0.00	0.00	n/a
client_total	total_mismatch	0.74	1.00	1.00	D0
client_total	rounding_shave	0.00	0.00	0.00	n/a
upstream	honest	0.00	0.00	0.00	n/a
upstream	under_report_output_50	0.00	0.00	0.00	n/a
upstream	under_report_output_90	0.00	0.00	0.00	n/a
upstream	under_report_input_50	0.00	0.00	0.00	n/a
upstream	drop_reasoning	0.00	0.00	0.00	n/a
upstream	inflate_cached	0.00	0.00	0.00	n/a
upstream	total_mismatch	0.00	0.00	0.00	n/a
upstream	rounding_shave	0.00	0.00	0.00	n/a
server_recount	honest	0.00	0.00	0.00	n/a
server_recount	under_report_output_50	0.00	0.00	0.00	n/a
server_recount	under_report_output_90	0.00	0.00	0.00	n/a
server_recount	under_report_input_50	0.00	0.00	0.00	n/a
server_recount	drop_reasoning	0.00	0.00	0.00	n/a
server_recount	inflate_cached	0.00	0.00	0.00	n/a
server_recount	total_mismatch	0.00	0.00	0.00	n/a
server_recount	rounding_shave	0.00	0.00	0.00	n/a
hybrid_reconcile	honest	0.00	0.00	0.00	n/a
hybrid_reconcile	under_report_output_50	0.00	0.00	0.00	n/a
hybrid_reconcile	under_report_output_90	0.00	0.00	0.00	n/a
hybrid_reconcile	under_report_input_50	0.00	0.00	0.00	n/a
hybrid_reconcile	drop_reasoning	0.00	0.00	0.00	n/a
hybrid_reconcile	inflate_cached	0.00	0.00	0.00	n/a

Table 17: M2: leakage efficiency vs concurrency (attacker arm; volume held constant).

architecture	c=1	c=5	c=20	c=50	c=100
client	0.228	0.228	0.228	0.228	0.228
client_logged	0.228	0.228	0.228	0.228	0.228
client_total	0.257	0.257	0.257	0.257	0.257
hybrid_reconcile	0.000	0.000	0.000	0.000	0.000
server_recount	0.000	0.000	0.000	0.000	0.000
upstream	0.000	0.000	0.000	0.000	0.000

Table 18: M2 as a transformation on the usage vector. Whether a manipulation pays off is determined by whether the pricing basis reads the field T changes.

architecture	pricing basis	manipulation T	vector change T(U)	leak eff.	why it succeeds / fails
client	category	honest	unchanged	+0.000	T is identity (honest)
client	category	under_report_output_50	output:60->30, total:98->68	+0.324	T lowers a category the basis prices
client	category	under_report_output_90	output:60->6, total:98->44	+0.583	T lowers a category the basis prices
client	category	under_report_input_50	input:8->4, total:98->94	+0.014	T lowers a category the basis prices
client	category	drop_reasoning	reasoning:30->0, total:98->68	+0.324	T lowers a category the basis prices
client	category	inflate_cached	output:60->30, cached_input:0->30	+0.313	T lowers a category the basis prices
client	category	total_mismatch	total:98->24	+0.000	T changes only the declared total; category basis never reads it
client	category	rounding_shave	input:8->7, output:60->57, total:98->94	+0.036	T lowers a category the basis prices
client_total	flat-total	honest	unchanged	-0.058	T is identity (honest)
client_total	flat-total	under_report_output_50	output:60->30, total:98->68	+0.266	T lowers the declared total the basis prices
client_total	flat-total	under_report_output_90	output:60->6, total:98->44	+0.525	T lowers the declared total the basis prices
client_total	flat-total	under_report_input_50	input:8->4, total:98->94	-0.014	T's change is priced at (near) the same rate
client_total	flat-total	drop_reasoning	reasoning:30->0, total:98->68	+0.266	T lowers the declared total the basis prices
client_total	flat-total	inflate_cached	output:60->30, cached_input:0->30	-0.058	T changes only subtotals; flat-total basis never reads them
client_total	flat-total	total_mismatch	total:98->24	+0.741	T lowers the declared total the basis prices
client_total	flat-total	rounding_shave	input:8->7, output:60->57, total:98->94	-0.014	T's change is priced at (near) the same rate

Table 19: Economic sensitivity. Dollar columns are SCENARIO values under synthetic price tiers; leakage efficiency and leaked tokens are the price-independent measures. Efficiency is invariant under uniform price scaling (low->medium) but not under a price-structure change (medium->high alters the output/input ratio).

architecture	manipulation	low (\$)	medium (\$)	high (\$)	leaked tokens	leak eff.	inv. under x10 scale?	d(ef) med->high
client	honest	+0.0000	+0.0000	+0.0000	0	+0.000	yes	+0.000
client_total	honest	-0.0160	-0.1600	-1.9200	-113	-0.058	yes	-0.012
client	under_report_output_50	+0.0900	+0.9000	+9.0000	635	+0.324	yes	+0.004
client_total	under_report_output_50	+0.0740	+0.7400	+7.0800	522	+0.266	yes	-0.009
client	under_report_output_90	+0.1620	+1.6200	+16.2000	1142	+0.583	yes	+0.007
client_total	under_report_output_90	+0.1460	+1.4600	+14.2800	1029	+0.525	yes	-0.006
client	under_report_input_50	+0.0040	+0.0400	+0.2400	28	+0.014	yes	-0.006
client_total	under_report_input_50	-0.0040	-0.0400	-0.7200	-28	-0.014	yes	-0.012
client	drop_reasoning	+0.0900	+0.9000	+9.0000	635	+0.324	yes	+0.004
client_total	drop_reasoning	+0.0740	+0.7400	+7.0800	522	+0.266	yes	-0.009
client	inflate_cached	+0.0870	+0.8700	+8.8200	613	+0.313	yes	+0.008
client_total	inflate_cached	-0.0160	-0.1600	-1.9200	-113	-0.058	yes	-0.012
client	total_mismatch	+0.0000	+0.0000	+0.0000	0	+0.000	yes	+0.000
client_total	total_mismatch	+0.2060	+2.0600	+20.2800	1452	+0.741	yes	-0.003
client	rounding_shave	+0.0100	+0.1000	+0.9600	71	+0.036	yes	-0.001
client_total	rounding_shave	-0.0040	-0.0400	-0.7200	-28	-0.014	yes	-0.012

Table 20: M1 ablation: abort-inclusive finalization closes the integrity violation; reservation independently provides the solvency (funds) guarantee. Both are needed for a correct design.

architecture	reservation	abort-finalization	worst leak/req	integrity	served (budget 2)	solvency
pre_debit	yes	n/a (pre-committed)	-0.0090	held	2/6	ok
reserve_reconcile	yes	yes	+0.0000	held	2/6	ok
reserve_refund_on_abort	yes	no	+0.0845	VIOLATED	6/6	ok
no_reserve_settle	no	yes	+0.0000	held	6/6	NEGATIVE
post_completion	no	no	+0.0845	VIOLATED	6/6	ok

Table 21: M2 ablation: performing a recount is not sufficient; it must be used as the billing basis. client_logged computes the recount and still leaks.

architecture	recount performed	recount used for billing	worst leak eff.	integrity	detection
client	no	no	0.595	VIOLATED	D0
client_logged	yes	no	0.595	VIOLATED	D1
client_total	no	no	0.745	VIOLATED	D0
server_recount	yes	yes	0.000	held	n/a
hybrid_reconcile	yes	yes	0.000	held	n/a
upstream	no	no	0.000	held	n/a

Table 22: Lifecycle failure injection. Every invariant violation occurs on a vulnerable architecture; the hardened architectures survive all injected failures, and a recount-engine failure is fail-closed.

id	failure injected	architecture	delivered	net debit	refund	leakage	invariant
F1	client disconnect mid-stream	post_completion	yes	0.0000	0.0000	0.2375	VIOLATED
F1	client disconnect mid-stream	reserve_reconcile	yes	0.2375	0.2175	0.0000	HELD
F2	client disconnect before first token	post_completion	yes	0.0000	0.0000	0.0275	VIOLATED
F2	client disconnect before first token	reserve_reconcile	yes	0.0275	0.4275	0.0000	HELD
F8	partial/truncated stream near completion	post_completion	yes	0.0000	0.0000	0.4325	VIOLATED
F8	partial/truncated stream near completion	reserve_reconcile	yes	0.4325	0.0225	0.0000	HELD
F9	audit read before settle grace period	post_completion	yes	0.0000	0.0000	0.2375	VIOLATED
F9	audit read before settle grace period	reserve_reconcile	yes	0.2375	0.2175	0.0000	HELD
F5	missing/unknown usage metadata	client	yes	0.6725	0.0000	0.0000	HELD
F5	missing/unknown usage metadata	server_recount	yes	0.6725	0.0000	0.0000	HELD
F7	duplicate usage metadata (replayed request_i	client	yes	0.2750	0.0000	0.3975	VIOLATED
F7	duplicate usage metadata (replayed request_i	server_recount	yes	0.6725	0.0000	0.0000	HELD
F6	delayed/out-of-order usage metadata	client	yes	0.4550	0.0000	0.2175	VIOLATED
F6	delayed/out-of-order usage metadata	server_recount	yes	0.6725	0.0000	0.0000	HELD
F4	tokenizer/recount engine unavailable	bench/server_recount	no	0.0000	0.0000	0.0000	HELD

Table 23: Architecture safety summary across all attacks and tiers.

mech	architecture	verdict	max req ASR	best detect
M1	pre_debit	safe	0.00	D3
M1	reserve_reconcile	safe	0.00	D3
M1	post_completion	LEAKS	1.00	D0
M1	reserve_refund_on_abort	LEAKS	1.00	D0
M2	client	LEAKS	1.00	D0
M2	client_logged	LEAKS	1.00	D1
M2	client_total	LEAKS	1.00	D0
M2	upstream	safe	0.00	D3
M2	server_recount	safe	0.00	D3
M2	hybrid_reconcile	safe	0.00	D3